

Project Presentation of IOHE Project
On

RoamNest

Team details :

Partha Pratim Tamuly(2210992031)

Mohit Kumar Gupta(2210991932)

Supervised By: Dr. Preeti Saini

Department of Computer Science and Engineering
Chitkara University, Punjab

Table of Contents



- Introduction
- Problem Statement
- Technical Details
- System Design
- Application Flow & Use Cases
- System Design
- Software & Hardware Requirements
- Objectives
- Project Highlights
- Key Features
- Conclusion
- Future Scope
- References/Links used



Fig.1

The sharing economy depends on trust, where platforms traditionally require users to share personal details before booking approval . However, studies show that early exposure of identity information often leads to discrimination and biased decisions . This creates an unfair system where subjective judgment overrides actual property availability . RoamNest addresses this issue by designing a system that removes bias from the booking process . The main objective is to implement a “Privacy-by-Design” approach that prioritizes objective factors over personal identity . The platform introduces a “Blind Booking” workflow, where sensitive user details are hidden until booking confirmation . It ensures trust through verification badges like ID, phone, and email verification instead of profile-based judgments .Key features include Role-Based Access Control (RBAC) for different user roles and a Progressive Disclosure Engine to protect user data . Additionally , the system supports dynamic property listings and an interactive, user-friendly interface for seamless experience.

Problem Statement

- Existing rental platforms in the sharing economy expose users' personal information—such as names, profile photos, and backgrounds—before a booking is confirmed. This early disclosure often leads to biased decision-making, where hosts may accept or reject requests based on subjective perceptions rather than objective criteria like availability or user credibility.
- As a result, many users face unfair rejection and unequal access to accommodation opportunities. This highlights a critical design flaw in current systems, where trust-building mechanisms unintentionally promote discrimination instead of ensuring fairness and inclusivity.

RoamNest is developed using a modern full-stack architecture combining a React.js frontend with a Spring Boot backend and PostgreSQL database. The frontend is built as a Single Page Application (SPA) using React and Vite, enabling fast rendering, dynamic UI updates, and smooth navigation. The backend is implemented using Spring Boot, which provides RESTful APIs to handle business logic, user authentication, and booking operations.

The system uses JSON Web Tokens (JWT) for secure, stateless authentication, ensuring scalability and performance. Role-Based Access Control (RBAC) is implemented to manage permissions for Admins, Owners, and Users. The database is designed using normalized relational models to maintain data integrity and prevent redundancy. Additionally, security features like password hashing (BCrypt), CORS configuration, and SQL injection prevention are integrated to ensure a secure and reliable application.

To understand the architecture of RoamNest, it is critical to understand the primary user journeys that the system supports.

User Registration and Authentication Flow

Users land on the RoamNest application and can choose to register as either a USER (Guest) or an OWNER (Host). Upon submission, the Spring Boot backend hashes the password using BCrypt and stores the profile in PostgreSQL. Upon logging in, the backend authenticates the credentials and issues a stateless JSON Web Token (JWT) valid for 24 hours. This token dictates what UI elements the React frontend displays.

Flow:

Start → Registration/Login → Select Role (Guest/Owner)

→ Guest Browses Properties → Apply Filters → Select Property

→ Send Booking Request (PENDING)

→ Owner Views Request (Blind Info)

→ Approve / Reject

→ If Approved → Reveal Guest Details

→ Booking Confirmed → End

Application Flow & Use Cases



Owner Property Listing Flow:

A user with the OWNER role accesses their dedicated dashboard. They can create a new property listing by providing a title, description, location, maximum guest capacity, and price per night. The data is securely transmitted via a POST request with the JWT in the Authorization header.

Guest Booking Flow:

A user with the USER role browses the platform, utilizing filters (location, price, dates). Upon selecting a property, they initiate a booking request. The system verifies property availability for the selected dates and generates a PENDING booking record in the database.

The "Blind Booking" Owner Decision Flow:

This is the core differentiating flow of the project.

1. The Owner receives a notification of a new booking request.
2. The Owner navigates to their OwnerBookings dashboard.
3. The backend API (/api/bookings/owner) retrieves the bookings but deliberately maps the data to a specialized DTO (Data Transfer Object) that hides the guest's name and contact info. Instead of "John Doe wants to book", the API sends "Guest RN-1024 wants to book for 3 nights (ID Verified)".
4. The Owner decides strictly based on their calendar availability and the guest's verified status.
5. Once the Owner clicks "Approve", the database state changes to APPROVED, and only then does the backend progressively disclose the guest's real identity to facilitate check-in logistics.

Architecture Overview

RoamNest is built using a modern Client-Server Single Page Application (SPA) architecture.

- **The Client (Frontend):** Developed entirely in React.js. It manages the user interface, client-side routing (React Router), state management, and form validations. It communicates with the backend exclusively via asynchronous HTTP requests using Axios.
- **The Server (Backend):** Developed in Spring Boot. It acts as a strict RESTful API provider. It contains no view-rendering logic (no JSP or Thymeleaf). It strictly receives JSON, processes business rules, interacts with the database, and returns JSON.
- **The Database:** A PostgreSQL relational database ensuring ACID compliance, crucial for handling concurrent booking transactions without double-booking a property.

Role-Based Access Control (RBAC)

System The system implements a rigid RBAC matrix.

- **ADMIN:** Has overarching system viewing capabilities.
- **OWNER:** Can POST to /api/properties, GET their specific booking queue, and PATCH booking statuses. Cannot make bookings.
- **USER:** Can GET properties, POST to /api/bookings, and GET their own booking history. Cannot create properties or approve bookings.

Software Requirements:

Component	Technology Used	Version
Runtime Environment	Node.js, Java Development Kit (JDK)	Node v18+, Java 17
Frontend Framework	React.js via Vite Build Tool	React 19, Vite 8
Backend Framework	Spring Boot	4.0.4
Database Management	PostgreSQL	15+
Security Architecture	Spring Security, jjwt-api	Latest
API Testing	Postman	Latest
Version Control	Git & GitHub	N/A
IDE	Visual Studio Code, IntelliJ IDEA	Latest

Software & Hardware Requirements

Component	Specification
Processor	Dual-core 2 GHz or higher
RAM	Minimum 8 GB
Storage	10 GB free disk space
Network	Stable broadband internet connection
Browser	Google Chrome / Firefox / Edge (latest)

Objectives



- **Eliminate Booking Bias** – Ensure fair decision-making by hiding personal identity during booking requests.
- **Ensure Privacy-by-Design** – Protect sensitive user data during the initial booking phase.
- **Implement Role-Based Access** – Provide controlled access for Admins, Owners, and Users.
- **Enable Efficient Booking System** – Manage booking requests with clear status tracking (Pending, Approved, Rejected).
- **Simplify Property Management** – Allow owners to easily add, update, and manage listings.
- **Enhance Security** – Use JWT authentication, password hashing, and secure APIs.
- **Improve User Experience** – Deliver a responsive and interactive UI for smooth navigation.
- **Build Scalable Architecture** – Use modern tech stack for performance and future growth.

Sign-Up Page



Create your RoamNest account

Sign up as a guest or owner for the live demo.

FULL NAME

USERNAME

PASSWORD

ROLE

Guest / User



PHONE NUMBER

9999999999

ADDRESS

India

Create Account

Already have an account? [Sign In](#)

Login Page

 **RoamNest**

Welcome back! Please login to your account.

USERNAME

PASSWORD

Sign In

Demo Accounts:
admin / admin | owner / owner | user / user

New to RoamNest? [Create an account](#)

User Dashboard

CHITKARA
UNIVERSITY



R RoamNest

demo_user_isha



Find Your Next Adventure

Discover unique homes and experiences around the world.

Available Stays

Search & Filter



Backwater Homestay in Lonavala #180

Lonavala, Maharashtra

₹10,050 / night

Up to 5 guests



partha's little nest

banur

₹3,000 / night

Up to 2 guests



Seaside Cottage

Goa

₹15,000 / night

Up to 4 guests

★ 5.0



Nagarkot Sunrise Villa

Nagarkot

₹7,500 / night

Up to 6 guests

Profile Page



Isha Verma

USER

@demo_user_isha

6 **4** **4.0**
TRIPS REVIEWS RATING

About Me Past Trips

Isha Verma

USER account

ACCOUNT DETAILS

USERNAME

demo_user_isha

FULL NAME

Isha Verma

PHONE NUMBER

+91 98765 10005

HOME ADDRESS

Sector 62, Noida, Uttar Pradesh

REVIEWS I'VE WRITTEN

4 reviews written

Heritage Haveli in Agra #171

Agra, Uttar Pradesh - 2/3/2026

"Great location and comfortable for a weekend trip."

★★★★☆

Owner DashBoard

 **RoamNest**

demo_owner_nisha Profile Logout

Manage Your Properties

List your homes, manage bookings, and grow your earnings.

Owner Dashboard

+ Add New Property

List a new home for guests to discover.

Create Listing

Booking Requests

No pending requests right now.

View Requests

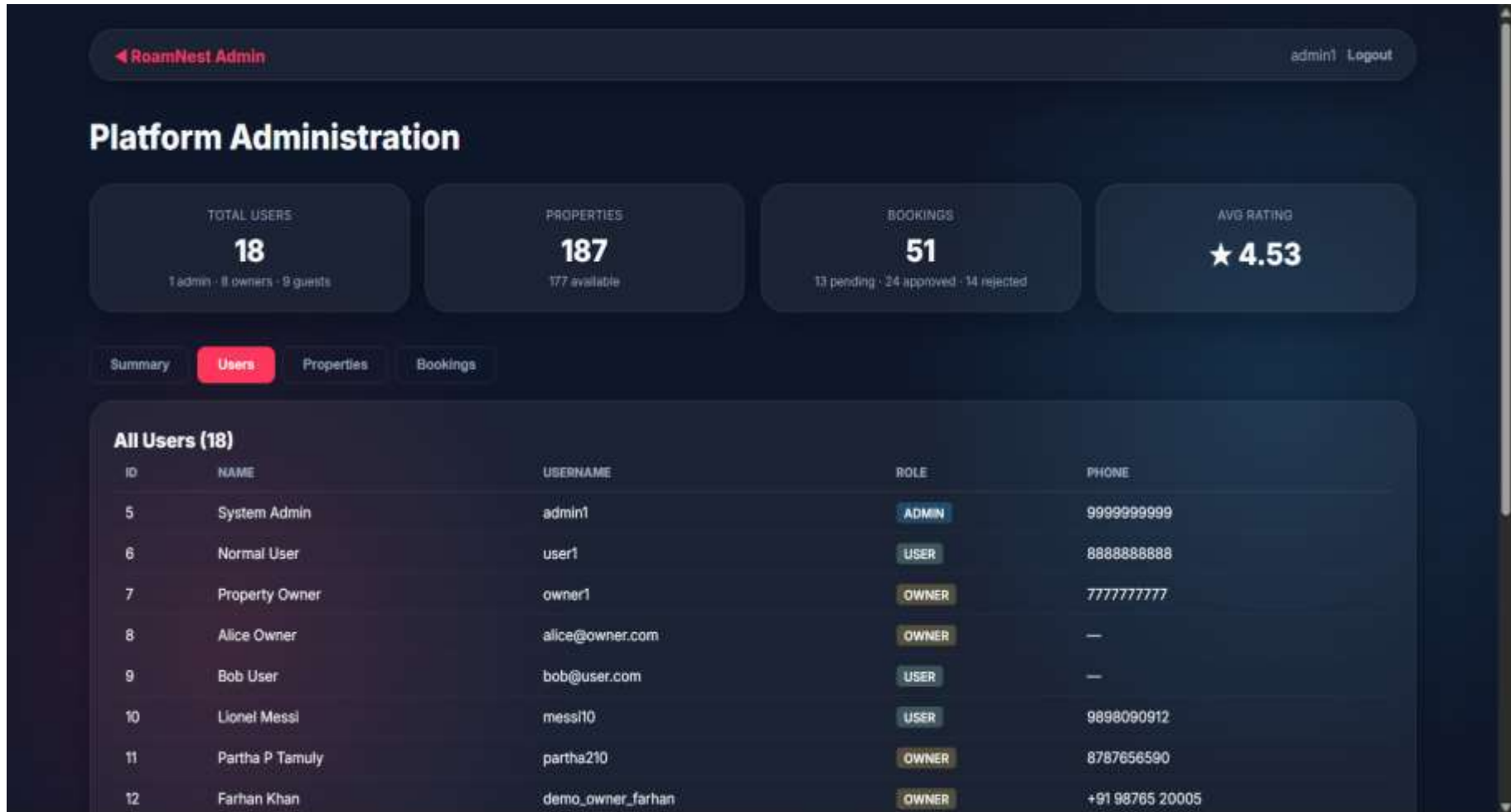
Messages

Your Properties

Add Property



Admin DashBoard



Key Features



- **User Authentication** – Secure sign up & login using email/password with JWT-based authentication.
- **Role-Based Access Control (RBAC)** – Separate access levels for Admin, Owner, and Guest users.
- **Blind Booking System** – Hides user identity during booking to prevent bias and discrimination.
- **Progressive Disclosure Engine** – Reveals user details only after booking approval.
- **Property Management** – Owners can add, edit, and manage multiple property listings.
- **Smart Search & Filters** – Users can search properties based on location, price, and preferences.
- **Booking Management** – Handles booking requests with status like Pending, Approved, or Rejected.
- **Secure System Design** – Password hashing, JWT authentication, and SQL injection protection.
- **Interactive UI** – Fast and responsive React-based Single Page Application (SPA).
- **Responsive Design** – Works smoothly across desktop and mobile devices.

Future Scope

- **Dynamic Pricing Engine:** Implementation of an AI-based algorithm that suggests or automatically adjusts property pricing based on local demand, seasonality, and local events.
- **Automated Moderation:** Integration of a robust review moderation system utilizing Natural Language Processing (NLP) to automatically detect and flag toxic, discriminatory, or biased feedback in property reviews.
- **Real-Time WebSockets:** Upgrading the message center and booking notifications from HTTP requests to persistent WebSockets for instant, real-time communication between hosts and guests.
- **KYC Integration:** Full integration with external background check APIs to issue highly trusted verified identity badges, further removing the need for profile photos as trust proxies.
- **Mobile Application:** Expanding the platform ecosystem by developing a dedicated native mobile application for iOS and Android utilizing React Native, sharing the same RESTful Spring Boot backend

Conclusion

RoamNest demonstrates a robust technical solution to a profound social issue— discrimination within the sharing economy. By implementing a "Blind Booking" Progressive Disclosure Protocol, the system successfully shields users from unconscious bias during the critical decision-making phase of renting.

Powered by a modern, high-performance stack utilizing React.js, Spring Boot, and PostgreSQL, the application enforces ethical design directly at the architectural level. RoamNest effectively proves that safety, trust, and complete anonymity can coexist, offering a scalable blueprint for the next generation of inclusive and fair peer-to-peer marketplaces.

References

Spring Boot Reference Documentation — <https://spring.io/projects/spring-boot>

React.js Official Documentation — <https://react.dev>

PostgreSQL Official Documentation — <https://www.postgresql.org/docs/>

JSON Web Tokens (JWT) Introduction and Debugger — <https://jwt.io/>

"Racial Discrimination in the Sharing Economy: Evidence from a Field Experiment" — Edelman, B., Luca, M., & Svirsky, D. (American Economic Journal: Applied Economics).

Vite Next Generation Frontend Tooling — <https://vitejs.dev/>

Spring Security Reference Architecture — <https://docs.spring.io/springsecurity/reference/>

React Router v7 Documentation — <https://reactrouter.com/>

"Privacy by Design" Principles — Ann Cavoukian, Ph.D. [10] Axios Promise based HTTP client for the browser and node.js — <https://axioshttp.com/>



**THANK
YOU**